

Algoritmica Grafurilor - Cursul 10

11 dicembre 2020

- 1 **Fluxuri în rețele**
 - Fluxuri de cost minim
- 2 **Reduceri în timp polinomial pentru probleme pe grafuri**
 - Mulțimii stabile de cardinal maxim
 - Colorări ale nodurilor
 - Probleme Hamiltoniene
- 3 **Exerciții pentru seminarul din săptămâna viitoare**
- 4 **Exerciții rezolvate (parțial)**

Să presupunem că în rețeaua $R = (G, s, t, c)$, se dă în plus o funcție de cost: $a : E \rightarrow \mathbb{R}; \forall ij \in E, a(ij) = a_{ij}$ este costul arcului ij (interpretat ca fiind costul trimiterii unei "unități" de flux pe arcul ij).

Dacă x este un flux în R , atunci costul lui x este

$$a(x) = \sum_{i,j} a_{ij} x_{ij}.$$

Problema fluxului de cost minim

Date: R o rețea, $a : E \rightarrow \mathbb{R}$ o funcție de cost, și $v \in \mathbb{R}_+$,

Determină: un flux x^0 în R astfel încât

$$a(x^0) = \min \{a(x) : x \text{ flux în } R, v(x) = v\}.$$

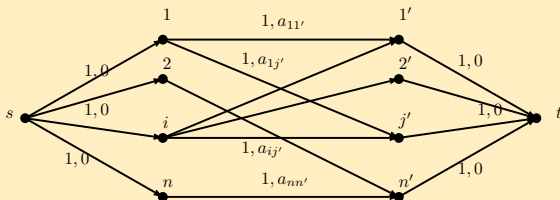
Să observăm că, dacă v nu este mai mare decât valoarea fluxului maxim în R , atunci problema are întotdeauna soluții ($a(x)$ este o funcție liniară definită pe mulțimea nevidă și compactă din $\mathbb{R}^{m+1}$ a tuturor fluxurilor de valoare v).

Fluxuri de cost minim - Exemple

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

1. Problema asignării. Se dau n muncitori și n slujbe. Costul repartizării muncitorului i slujbei j este a_{ij} . Să se asigneze fiecare muncitor câte unei slujbe astfel încât costul total să fie minim.

Considerăm rețeaua bipartită de mai jos, unde fiecare arc este etichetat cu capacitatea sa urmată de cost. Astfel, $c_{ij'} = 1$, $c_{si} = 1$, $a_{si} = 0$, $c_{j't} = 1$, și $a_{j't} = 0$, $\forall i, j \in \{1, 2, \dots, n\}$.



Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Un flux de cost minim de valoare n constituie soluția problemei.

Similar putem determina un cuplaj perfect de pondere minimă într-un graf bipartit.

2. Problema de transport Hitchcock-Koopmans. Un produs, disponibil în depozitele $D_1, \dots, D_n$ în cantitățile $d_1, \dots, d_n$, respectiv, este cerut către clienții $C_1, \dots, C_m$ în cantitățile $c_1, \dots, c_m$, respectiv. Se cunosc costurile de transport unitar, a_{ij} - de la depozitul D_i la clientul C_j , $\forall i \in \{1, \dots, n\}, \forall j \in \{1, \dots, m\}$.

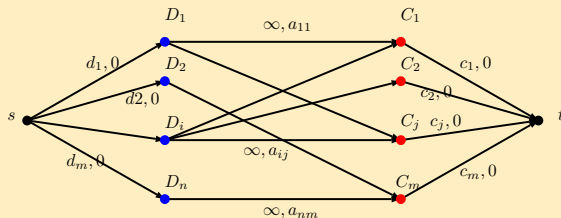
Să se determine o schemă de transport care să satisfacă cererea clienților cu un cost total al transportului minim.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Fluxuri de cost minim - Exemple

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Problema are soluție numai dacă $\sum_{i=1}^n d_i \geq \sum_{j=1}^m c_j$. În acest caz, un flux de cost minim de valoare $v = \sum_{i=1}^m c_i$, în rețeaua de mai jos, rezolvă problema.



Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Definiție

Fie x un flux în $R = (G, s, t, c)$ și $a : E \rightarrow \mathbb{R}$ o funcție de cost.

- Dacă P este un A -drum în R relativ la x , atunci **costul drumului** P este definit ca

$$a(P) = \sum_{ij \in E(P), ij \text{ forward}} a_{ij} - \sum_{ij \in E(P), ji \text{ backward}} a_{ji}$$

- Dacă C este un A -drum închis în R relativ la x , atunci $a(C)$ este calculat ca mai sus, după stabilirea unui sens de parcurgere alui C (este posibil ca ambele sensuri să ofere A -drumuri relativ la x).

Remarci

- Dacă P este un drum de creștere relativ la x , atunci $x^1 = x \otimes r(P)$ este un flux de valoare $v(x^1) = v(x) + r(P)$ și cost $a(x^1) = a(x) + r(P) \cdot a(P)$.

Fluxuri de cost minim

Remarci

- Dacă C este un A -drum închis în R relativ la x , atunci $x^1 = x \otimes r(C)$ este un flux de valoare $v(x^1) = v(x)$ și cost $a(x^1) = a(x) + r(C) \cdot a(C)$. Urmează că dacă $a(C) < 0$ atunci x^1 este un flux de aceeași valoare cu x dar de cost $a(x^1) < a(x)$.

Teorema 1

Un flux x de valoare v este un flux de cost minim dacă și numai dacă nu există un A -drum închis de cost negativ relativ la x în R .

Demonstrație. " $\Rightarrow$ " Din remarca de mai sus.

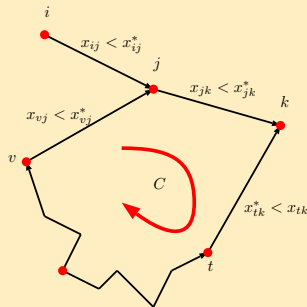
" $\Leftarrow$ " Fie x un flux de valoare v astfel încât nu există un A -drum închis de cost negativ relativ la x în R . Fie x^* un flux de cost minim de valoare v astfel încât

$$\Delta(x, x^*) = \min \{ \Delta(x, x') : x' \text{ flux de cost minim de valoare } v \},$$

unde $\Delta(x, x') = |\{ij \in E : x_{ij} \neq x'_{ij}\}|$.

Dacă $\Delta(x, x^*) = 0$, atunci $x = x^*$, astfel x este un flux de cost minim. Altfel, $\Delta(x, x^*) > 0$ și există ij astfel încât $x_{ij} \neq x_{ij}^*$. Să presupunem că $0 \leq x_{ij} < x_{ij}^* \leq c_{ij}$ (dacă $x_{ij} > x_{ij}^*$, raționamentul este similar). Din legea conservării fluxului, există $jk \in E$ astfel încât $0 \leq x_{jk} < x_{jk}^* \leq c_{jk}$, sau există $kj \in E$ astfel încât $0 \leq x_{kj}^* < x_{kj} \leq c_{kj}$.

Deoarece numărul de noduri este finit, repetând acest raționament, obținem, C , un A -drum închis relativ la x în R :



Dacă parcurgem C în sens contrar, obținem un A -drum închis, C' , relativ la x^* . Deoarece $a(C) \geq 0$ (din ipoteză), și $a(C') = -a(C)$ urmează că $a(C) = 0$. (x^* este de cost minim; astfel, din implicația directă, $a(C') \geq 0$).

Dacă vom considera $x' = x^* \otimes \delta(C')$, unde

$$\delta(C') = \min \left\{ \min_{kj \text{ forward in } C'} (x_{kj} - x_{kj}^*), \min_{kj \text{ backward in } C'} (x_{jk}^* - x_{jk}) \right\},$$

atunci x' satisface $v(x') = v(x^*) = v$, $a(x') = a(x^*) + \delta(C') \cdot a(C') = a(x^*)$.

Astfel x' este un flux de cost minim de valoare v , dar $\Delta(x, x') < \Delta(x, x^*)$, în contradicție cu alegerea lui x^* . Astfel $\Delta(x, x^*) = 0$, și teorema este demonstrată. $\square$

Teorema 2

Dacă x este un flux de cost minim de valoare v și P_0 este un drum de creștere relativ la x astfel încât

$$a(P_0) = \min \{a(P) : P \text{ drum de creștere relativ la } x\},$$

atunci $x^1 = x \otimes r(P_0)$ este un flux de cost minim de valoare $v(x^1) = v + r(P_0)$.

Demonstrație. Omisă.

Un drum de creștere de cost minim poate fi găsit folosind un algoritm pentru drumuri de cost minim. Dacă x este un flux în R și $a : E \rightarrow \mathbb{R}$ este o funcție de cost, atunci luând $a_{ij} = \infty$ dacă $ij \notin E$ (când $x_{ij} = 0$), construim

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

$$\bar{a}_{ij} = \begin{cases} a_{ij}, & \text{dacă } x_{ij} < c_{ij} \text{ și } x_{ji} = 0, \\ \min \{ a_{ij}, -a_{ji} \}, & \text{dacă } x_{ij} < c_{ij} \text{ și } x_{ji} > 0, \\ -a_{ji}, & \text{dacă } x_{ij} = c_{ij} \text{ și } x_{ji} > 0, \\ +\infty, & \text{dacă } x_{ij} = c_{ij} \text{ și } x_{ji} = 0. \end{cases}$$

Un *st*-drum de cost $\bar{a}$ minim corespunde unui drum de creștere de cost minim relativ la x în R , și un circuit de cost negativ corespunde unui *A*-drum închis relativ la x în R .

Atunci, avem următorul algoritm pentru determinarea unui flux de cost minim:

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Fluxuri de cost minim - Un algoritm generic (Klein, Busacker, Gowan etc.)

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

fie x un flux de valoare $v' \leq v$; // x poate fi nul sau $x = (v/v(y)) \cdot y$, unde y este un flux valoare maximă.

while ($\exists C$ un circuit de cost $\bar{a}$ negativ) do

$x \leftarrow x \otimes r(C)$;

end while

while ($v(x) < v$) do

determină un st -drum P de cost $\bar{a}$ minim;

$x \leftarrow x \otimes \min\{r(P), v - v(x)\}$;

end while

Complexitatea timp al celui de-al doilea **while** este $\mathcal{O}(n^3 v)$ (dacă pornim cu fluxul nul și capacitățile sunt întregi). Primul **while** poate fi implementat astfel încât să aibă $\mathcal{O}(nm^2 \log n)$ iterații.

Memento

- Fie $P_i : I_i \rightarrow \{da, nu\}$ ($i \in \{1, 2\}$) două probleme de decizie.
 P_1 se reduce polinomial la P_2 , și notăm aceasta prin $P_1 \leq_P P_2$, dacă există o funcție calculabilă în timp polinomial $\Phi : I_1 \rightarrow I_2$, astfel încât $P_1(i) = P_2(\Phi(i))$, $\forall i \in I_1$.
- Funcțiile Φ vor fi definite folosind un algoritm care construiește, pentru fiecare instanță $i_1 \in I_1$, o instanță $i_2 \in I_2$ în timp polinomial (în dimensiunea lui i_1), astfel încât $P_1(i_1) = da$ dacă și numai dacă $P_2(i_2) = da$.
- Construcția din spatele reducerii în timp polinomial arată cum prima problemă poate fi rezolvată eficient folosind un oracol pentru cea de-a doua.

- Relația $\leq_P$ este o relație tranzitivă pe mulțimea problemelor de decizie (deoarece, clasa funcțiilor polinomial calculabile este închisă la compunere).

Exemplu

SAT $\leq_P$ 3SAT

SAT

Instantă: $U = \{u_1, u_2, \dots, u_n\}$ o mulțime finită de variabile booleene;

$C = C_1 \wedge C_2 \dots \wedge C_m$ o formulă CNF peste U :

$C_i = v_{i1} \vee v_{i2} \vee \dots \vee v_{i_{k_i}}, i = \overline{1, m}$, unde

$\forall i_j, \exists \alpha \in \{1, 2, \dots, n\}$ a. î. $v_{i_j} = u_\alpha$ sau $v_{i_j} = \overline{u_\alpha}$.

Întrebare: Există o asignare $t : U \rightarrow \{true, false\}$ a. î. $t(C) = true$?

Reduceri în timp polinomial - Problema mulțimii stabile de cardinal maxim

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

SM

Instanță: $G = (V, E)$ un graf și $k \in \mathbb{N}$.

Întrebare: Există o mulțime stabilă S în G astfel încât $|S| \geq k$?

C. Croitoru - Graph Algorithms C. Croitoru - Graph Algorithms C. Croitoru - Graph

Teorema 3

(Karp, 1972). $3SAT \leq_P SM$.

C. Croitoru - Graph Algorithms C. Croitoru - Graph Algorithms C. Croitoru - Graph

Demonstrație. Fie $U = \{u_1, u_2, \dots, u_n\}$, ($n \in \mathbb{N}^*$), $C = C_1 \wedge C_2 \dots \wedge C_m$ ($m \in \mathbb{N}^*$) cu $C_i = v_{i1} \vee v_{i2} \vee v_{i3}$, $i = \overline{1, m}$, (unde $\forall i_j, \exists \alpha \in \{1, 2, \dots, n\}$ a. î. $v_{ij} = u_\alpha$ sau $v_{ij} = \overline{u}_\alpha$) reprezentând datele unei instanțe a problemei 3SAT.

Vom construi în timp polinomial (în $n + m$) un graf G și $k \in \mathbb{N}$, astfel încât există o asignare t care satisface C dacă și numai dacă există o mulțime stabilă S în G astfel încât $|S| \geq k$.

Reduceri în timp polinomial - Problema mulțimii stabile de cardinal maxim

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Demonstrație (continuare).

Construcția grafului G :

- $\forall i \in \{1, 2, \dots, n\}$, fie grafurile disjuncte $T_i = (\{u_i, \bar{u}_i\}, \{u_i \bar{u}_i\})$.
- $\forall j \in \{1, 2, \dots, m\}$, fie grafurile disjuncte $Z_j = (\{a_{j1}, a_{j2}, a_{j3}\}, \{a_{j1} a_{j2}, a_{j2} a_{j3}, a_{j3} a_{j1}\})$.
- $\forall j \in \{1, 2, \dots, m\}$, fie mulțimea de muchii $E_j = \{a_{j1} v_{j1}, a_{j2} v_{j2}, a_{j3} v_{j3}\}$, unde $v_{j1} \vee v_{j2} \vee v_{j3}$ este clauza C_j .

$$V(G) = \left(\bigcup_{i=1}^n V(T_i) \right) \cup \left(\bigcup_{j=1}^m V(Z_j) \right)$$

$$E(G) = \left(\bigcup_{i=1}^n E(T_i) \right) \cup \left(\bigcup_{j=1}^m E(Z_j) \cup E_j \right).$$

Reduceri în timp polinomial - Problema mulțimii stabile de cardinal maxim

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Evident, construcția lui G se poate face în timp polinomial relativ la dimensiunea $n + m$ a instanței 3SAT. Fie $k = n + m$.

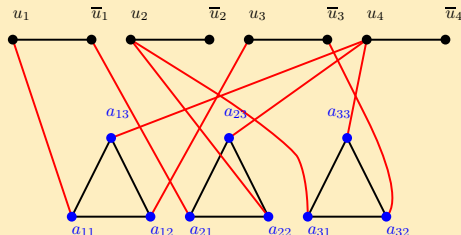
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exemplu

$U = \{u_1, u_2, u_3, u_4\}$; $C = (u_1 \vee u_3 \vee u_4) \wedge (\bar{u}_1 \vee u_2 \vee u_4) \wedge (u_2 \vee \bar{u}_3 \vee u_4)$;
 $k = 4 + 3 = 7$.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms



Reduceri în timp polinomial - Problema mulțimii stabile de cardinal maxim

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Să presupunem că răspunsul la întrebarea problemei SM pentru (G, k) instanță este da.

$\exists S \in \mathcal{S}_G$ (familia tuturor mulțimilor stabile din G) astfel încât $|S| \geq k$. Deoarece fiecare mulțime stabilă poate avea cel mult un nod în fiecare $V(T_i)$ și fiecare $V(Z_j)$, urmează că $|S| = k$ și $|S \cap V(T_i)| = 1$, $|S \cap V(Z_j)| = 1$, $\forall i = \overline{1, n}$, $\forall j = \overline{1, m}$.

Fie $t : U \rightarrow \{true, false\}$ dată prin

$$t(u_i) = \begin{cases} true, & \text{dacă } S \cap V(T_i) = \{\overline{u_i}\} \\ false, & \text{dacă } S \cap V(T_i) = \{u_i\} \end{cases}$$

Atunci, $t(C_j) = true$, $\forall j = \overline{1, m}$, astfel $t(C) = true$, și răspunsul la 3SAT este da.

Într-adevăr, $\forall j = \overline{1, m}$, dacă $C_j = v_{j1} \vee v_{j2} \vee v_{j3}$ și $S \cap V(Z_j) = \{a_{jk}\}$ ($k \in \{1, 2, 3\}$), atunci (deoarece $a_{jk}v_{jk} \in E$) urmează că $v_{jk} \notin S$.

Reduceri în timp polinomial - Problema mulțimii stabile de cardinal maxim

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- Dacă $v_{jk} = u_\alpha$, atunci $u_\alpha \notin S$, deci $\bar{u}_\alpha \in S$, și - din definiția lui t - avem $t(u_\alpha) = \text{true}$, adică, $t(v_{jk}) = \text{true}$, ceea ce implică $t(C_j) = \text{true}$.
- Dacă $v_{jk} = \bar{u}_\alpha$, atunci $\bar{u}_\alpha \notin S$, deci $u_\alpha \in S$, și - din definiția lui t - avem $t(\bar{u}_\alpha) = \text{true}$, adică, $t(v_{jk}) = \text{true}$, ceea ce implică $t(C_j) = \text{true}$.

Reciproc, dacă răspunsul la întrebarea problemei 3SAT este da, atunci $\exists t : U \rightarrow \{\text{true}, \text{false}\}$ astfel încât $t(C_j) = \text{true}, \forall j = \overline{1, m}$.

Fie $\bar{S}_1$ mulțimea stabilă $\bar{S}_1 = \bigcup_{i=1}^n V'_i$ cu n noduri, unde

$$V'_i = \begin{cases} \{\bar{u}_i\}, & \text{dacă } t(u_i) = \text{true} \\ \{u_i\}, & \text{dacă } t(u_i) = \text{false} \end{cases}$$

Reduceri în timp polinomial - Problema mulțimii stabile de cardinal maxim

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Atunci, deoarece $t(C_j) = true$, $\forall j = \overline{1, m}$, urmează că există $k_j \in \{1, 2, 3\}$ astfel încât $t(v_{jk_j}) = true$. Fie $\overline{S}_2 = \bigcup_{j=1}^m \{a_{jk_j}\}$. Evident, $\overline{S}_2$ este

o mulțime stabilă în G având m noduri.

Fie $\overline{S} = \overline{S}_1 \cup \overline{S}_2$. Evident, $|\overline{S}| = n + m = k$ (astfel $|\overline{S}| \geq k$). Dacă arătăm că $\overline{S}$ este o mulțime stabilă, atunci **răspunsul la SM pentru instanță (G, k) este da.**

Să presupunem că $\exists v, w \in \overline{S}$ astfel încât $e = vw \in E(G)$. Atunci o extremitate a lui e este în $\overline{S}_1$, iar cealaltă în $\overline{S}_2$. Dacă $v \in \overline{S}_1$, atunci avem două cazuri:

- $v = u_\alpha$, $w = a_{jk_j}$, $\alpha \in \{1, 2, \dots, n\}$, $j \in \{1, 2, \dots, m\}$, $k_j \in \{1, 2, 3\}$ și $v_{jk_j} = u_\alpha$.

Deoarece $t(v_{jk_j}) = true$, urmează că $t(u_\alpha) = true$, astfel $v = u_\alpha \notin \overline{S}_1$, contradicție.

Reduceri în timp polinomial - Problema mulțimii stabile de cardinal maxim

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.
Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

- $v = \overline{u}_\alpha$, $w = a_{jk_j}$, $\alpha \in \{1, 2, \dots, n\}$, $j \in \{1, 2, \dots, m\}$, $k_j \in \{1, 2, 3\}$
 şı $v_{jk_j} = \overline{u}_\alpha$.

Deoarece $t(v_{jk_j}) = true$, urmează că $t(\bar{u}_\alpha) = true$, astfel $t(u_\alpha) = false$. Astfel, $v = \bar{u}_\alpha \notin \bar{S}_1$, contradicție. $\square$

O demonstrație similară poate fi făcută pentru a arăta că $\text{SAT} \leq_P \text{SM}$, singura diferență este că Z_i sunt grafuri complete cu k_i noduri.

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Reduceri în timp polinomial - Colorări ale nodurilor

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

COL

Instanță: $G = (V, E)$ graf și $p \in \mathbb{N}^*$.

Întrebare: Există o p -colorare (a nodurilor) lui G ?

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Teorema 4

$3SAT \leq_P COL$.

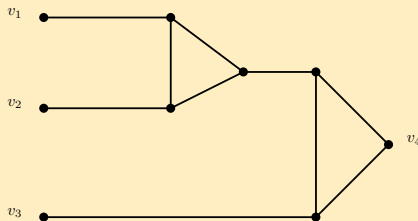
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Această teoremă arată că problema colorării nodurilor este NP-hard. Demonstrația dată mai jos arată că problema, care se obține din COL prin restricția la instanțele cu $p = 3$, care poate fi numită 3-COL, este NP-hard, de asemeni.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Lema 1

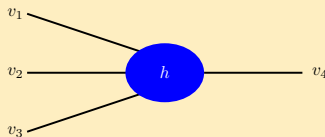
Fie H graful



- a) Dacă c este o 3-colorare a lui H a. î. $c(v_1) = c(v_2) = c(v_3) = a \in \{1, 2, 3\}$, atunci în mod necesar $c(v_4) = a$ (forcing).
- b) Dacă $c : \{v_1, v_2, v_3\} \rightarrow \{1, 2, 3\}$ satisface $c(\{v_1, v_2, v_3\}) \neq \{a\}$, ($a \in \{1, 2, 3\}$), atunci c poate fi extinsă la o 3-colorare c a lui H cu $c(v_4) \neq a$.

Demonstrație. Se examinează lista 3-colorărilor lui H .

Vom utiliza reprezentarea simplificată a grafului H :

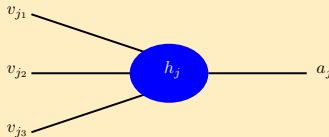


Demonstrația Teoremei 4. Considerăm datele unei instanțe a 3SAT: $U = \{u_1, \dots, u_n\}$, ($n \in \mathbb{N}^*$), o mulțime de variabile booleene, și $C = C_1 \wedge \dots \wedge C_m$, ($m \in \mathbb{N}^*$) formulă CNF cu $C_j = v_{j_1} \vee v_{j_2} \vee v_{j_3}$, $\forall j = \overline{1, m}$, unde $\forall i = \overline{1, 3}$, $\exists \alpha$ a. î. $v_{j_i} = u_\alpha$ sau $v_{j_i} = \overline{u}_\alpha$.

Vom construi un graf G cu proprietatea că G este 3-colorabil dacă și numai dacă răspunsul la 3SAT pentru această instanță este da, adică, există o asignare $t : U \rightarrow \{true, false\}$ astfel încât $t(C) = true$.

Construcția necesită un timp polinomial, și constă din următorii pași:

- Considerăm grafurile disjuncte (V_i, E_i) , $\forall i = \overline{1, n}$, unde $V_i = \{u_i, \overline{u_i}\}$ și $E_i = \{u_i \overline{u_i}\}$.
- Pentru $C_j = v_{j_1} \vee v_{j_2} \vee v_{j_3}$, $\forall j = \overline{1, m}$, considerăm grafurile:



unde, v_{j_k} ($k = \overline{1, 3}$) sunt nodurile corespunzătoare literalilor v_{j_k} , grafurile h_j sunt disjuncte, și a_j sunt noduri distincte.

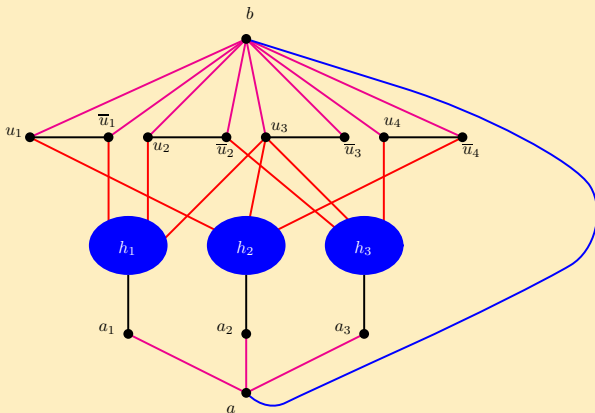
- Considerăm un nod nou a , și toate muchiile aa_j , $\forall j = \overline{1, m}$.
- Considerăm un nod nou b , toate muchiile bu_i , $b\overline{u_i}$, $\forall i = \overline{1, n}$ și ba .

Graful G are un număr de liniar de noduri relativ la $n + m$.

Exemplu

$U = \{u_1, u_2, u_3, u_4\}$, $C = (\bar{u}_1 \vee u_2 \vee u_3) \wedge (u_1 \vee u_3 \vee \bar{u}_4) \wedge (\bar{u}_2 \vee u_3 \vee u_4)$.

Graful G este



Să presupunem că răspunsul la 3SAT pentru instanță considerată este da

Astfel $\exists t : U \rightarrow \{true, false\}$ a. î. $t(C) = true$, adică, $t(C_j) = true$, $\forall j = \overline{1, m}$. Vom arăta că G este 3-colorabil.

Colorăm mai întâi nodurile u_i și $\overline{u_i}$, $\forall i \overline{1, n}$.

$$\begin{cases} c(u_i) = 1 \text{ și } c(\overline{u_i}) = 2, \text{ dacă } t(u_i) = true \\ c(u_i) = 2 \text{ și } c(\overline{u_i}) = 1, \text{ dacă } t(u_i) = false \end{cases}$$

Observăm că, dacă v este un literal, atunci $c(v) = 2$ dacă și numai dacă $t(v) = false$.

Deoarece t este o asignare pentru care satisface C , $t(C_j) = true$, $\forall j = \overline{1, m}$. Urmează că $c(\{v_{j_1}, v_{j_2}, v_{j_3}\}) \neq \{2\}$, $\forall j = \overline{1, m}$.

Din Lema 1 b), putem extinde c la o 3-colorare, în fiecare graf h_j , astfel încât $c(a_j) \neq 2$, adică $c(a_j) \in \{1, 3\}$.

Luând $c(a) = 2$ și $c(b) = 3$, obținem o 3-colorare a lui G .

Reduceri în timp polinomial - Colorări ale nodurilor

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Reciproc, să presupunem că G este 3-colorabil.

Putem presupune că $c(b) = 3$ și $c(a) = 2$ (altfel, redenumim culorile).
Urmează că $\{c(u_i), c(\bar{u}_i)\} = \{1, 2\}$, $\forall i = \overline{1, n}$ și $c(a_j) \in \{1, 3\}$, $\forall j = \overline{1, m}$.

Din Lema 1 a), urmează că $c(\{v_{j_1}, v_{j_2}, v_{j_3}\}) \neq 2$, $\forall j = \overline{1, m}$. Aceasta înseamnă că, $\forall j = \overline{1, m}$, există a $v_{j_k} \in C_j$ astfel încât $c(v_{j_k}) = 1$.

Astfel, definind $t : U \rightarrow \{true, false\}$ prin

$$t(u_i) = \begin{cases} true, & \text{dacă } c(u_i) = 1 \\ false, & \text{dacă } c(u_i) = 2 \end{cases}$$

Obținem o asignare cu proprietatea că $t(C_j) = true$, $\forall j = \overline{1, m}$.

Astfel răspunsul la 3SAT pentru instanța dată este da.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Memento: Fie G un (di)graf. Un circuit C al lui G este un circuit Hamiltonian dacă $V(C) = V(G)$.

Un drum deschis P al lui G este un **drum Hamiltonian** dacă $V(P) = V(G)$. Un **(di)graf Hamiltonian** este un (di)graf care are un circuit Hamiltonian. Un **(di)graf trasabil** este a (di)graf care conține un drum Hamiltonian.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Teorema 5

(Nash-Williams, 1969) Următoarele cinci probleme sunt echivalente polinomial:

CH: Dat un graf G . Este G Hamiltonian?

TR: Dat un graf G . Este G trasabil?

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

DCH: Dat un digraf G . Este G Hamiltonian?

DTR: Dat un digraf G . Este G trasabil?

BCH: Dat un graf bipartit G . Este G Hamiltonian?

Remarcă

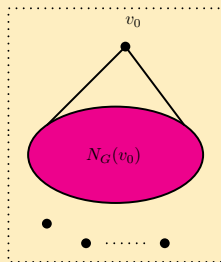
P_1 și P_2 sunt echivalente polinomial dacă $P_1 \leq_P P_2$ și $P_2 \leq_P P_1$.

Demonstrația Teoremei 5.

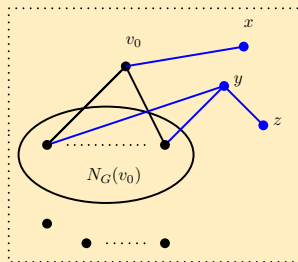
$CH \leq_P TR$ Fie G un graf și $v_0 \in V(G)$. Construim în timp polinomial un graf H astfel încât G este Hamiltonian dacă și numai dacă H este trasabil.

Fie $V(H) = V(G) \cup \{x, y, z\}$ și $E(H) = E(G) \cup \{xv_0, yz\} \cup \{wy : w \in V(G) \cap N_G(v_0)\}$.

Demonstrația Teoremei 5 (continuare).



G

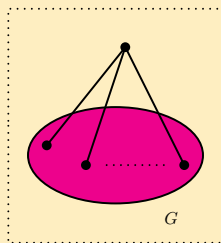


H

Atunci, H este trasabil dacă și numai dacă are un drum Hamiltonian, P , cu extremitățile x și z (care au gradul 1 în H). P există în H dacă și numai dacă în G există un drum Hamiltonian cu o extremitate în v_0 și cealaltă un vecin al lui v_0 , adică, dacă și numai dacă G este Hamiltonian.

Demonstrația Teoremei 5 (continuare).

$TR \leq_P CH$ Fie G un graf. Considerăm $H = G + K_1$. Atunci, H este Hamiltonian dacă și numai dacă G are un drum Hamiltonian.



$G + K_1$

Echivalența problemelor DCH și DTR se dovedește similar.

$CH \leq_P DCH$ Fie G un graf. Fie D digraful obținut din G prin înlocuirea fiecărei muchii cu o pereche simetrică de arce.

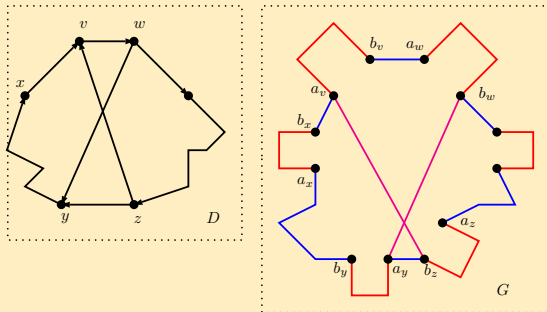
Reduceri în timp polinomial - Probleme Hamiltoniene

Evident fiecare circuit în G dă un circuit în D și reciproc, fiecare circuit în D provine dintr-un circuit în G .

$DCH \leq_P CH$ fie D un digraf. Fiecare nod $v \in V(D)$ este înlocuit printr-un graf neorientat $P_3(v)$ cu extremități a_v și b_v :

$$P_3(v) = (\{a_v, b_v, c_v, d_v\}, \{a_v c_v, c_v d_v, d_v b_v\}).$$

Fiecare arc $vw \in E(D)$ este înlocuit prin muchia (neorientată) $b_v a_w$. Fie G graful obținut (în timp polinomial) astfel:



C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Fiecare circuit C al lui D corespunde unui circuit în G și, reciproc, fiecare circuit în G corespunde unui circuit al lui D . Urmează că D este Hamiltonian dacă și numai dacă G este Hamiltonian.

Să observăm că dacă C este un circuit al lui G , atunci acesta este generat de un circuit C' al lui D , și $\text{length}(C) = 3 \cdot \text{length}(C') + \text{length}(C') = 4 \cdot \text{length}(C')$. Urmează că orice circuit al lui G este par, deci G este un graf bipartit.

Astfel demonstrația de mai sus ($\text{DCH} \leq_P \text{CH}$) este de fapt $\text{DCH} \leq_P \text{BCH}$.

Deoarece $\text{BCH} \leq_P \text{CH}$ este evidentă, Teorema 5 este complet demonstrată. $\square$

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul din săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 1. Arătați că următoarea problemă este NP-completă
INT

Instanță: $n, m \in \mathbb{N}^*$, $A \in \mathbb{Z}^{m \times n}$, $b \in \mathbb{Z}^m$.

Întrebare: Există o asignare $x \in \mathbb{Z}^n$ a. î. $Ax \leq b$?

(Inegalitatea $\leq$ dintre doi vectori este pe componente.) *Hint:* You can try $SM \leq_P$ INT.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exercițiul 2. Considerăm următoarea problemă de decizie
ACYCLIC

Instanță: $G = (V, E)$ un digraf și $p \in \mathbb{N}$.

Întrebare: Există $A \subseteq V$ astfel încât $|A| \leq p$ și $G - A$ nu conține circuite?

Arătați că $SM \leq_P$ ACYCLIC.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exercițiul 3. Un hipergraf k -uniform este o pereche $H = (V, E)$, unde $V \neq \emptyset$ este o mulțime finită, $k \in \mathbb{N}^* \setminus \{1\}$, și $E \subseteq \mathcal{P}_k(V) = \{A \subseteq V : |A| = k\}$. Este ușor de văzut că un hipergraf 2-uniform este un graf simplu.

Spunem că un hipergraf k -uniform $H = (V, E)$ este *simplu* dacă există o funcție $c : V \rightarrow \{1, 2, \dots, k\}$ astfel încât $\forall u, v \in V, u \neq v$, dacă $u, v \in e$ pentru o anumită muchie $e \in E$, atunci $c(u) \neq c(v)$. Considerăm următoarea problemă de decizie

k -SIMPLE

Instantă: H un hipergraf k -uniform.

Întrebare: H este simplu?

- (a) Arătați că problema 3-SIMPLE este NP-completă.
- (b) Arătați că problema 2-SIMPLE este în P.

Exercițiul 4. Considerăm următoarea problemă de decizie:

AGM

Instanță: G un graf, $k \in \mathbb{N}$.

Întrebare: G are un arbore parțial T astfel încât $\Delta(T) \geq k$?

Arătați că $\text{AGM} \in \text{P}$.

Exercițiul 5. Fie $D = (V, E)$ un digraf fără bucle. O mulțime stabilă a lui D , $S \subseteq V$, este numită **quasi-kernel** dacă fiecare nod $v \in V \setminus S$ sibil din S pe un drum de lungime cel mult 2.

- (a) Arătați că un quasi-kernel poate fi construit în $\mathcal{O}(n + m)$, unde $n = |V|$ și $m = |E|$.
- (b) Arătați că 3-SAT se poate reduce în timp polinomial la problema de determinării dacă într-un digraf dat există un quasi-kernel care conține un nod dat.

Exercițiul 6. Considerăm următoarea problemă de decizie: **LPL**

Instantă: G un graf, $k \in \mathbb{N}$.

Întrebare: Are G un drum P astfel încât $length(P) \geq k$?

Arătați că LPL este NP-completă.

Exercițiul 7. Un **kernel** într-un digraf $G = (V, E)$ este o mulțime stabilă $S \subseteq V$ astfel încât $\forall u \in V \setminus S$ există $v \in S$ cu $vu \in E$. Considerăm următoarea problemă de decizie:

KERNEL

Instantă: G un digraf.

Întrebare: Are G un kernel?

Arătați că următoarea construcție conduce la o reducere polinomială a lui SAT la KERNEL (i.e. $\text{SAT} \leq_P \text{KERNEL}$):

Exercițiul 7 (continuare). Pentru fiecare conjuncție de clauze, F , instanță a lui SAT, definim un digraf G (o instanță pentru NUCLEU):

- pentru fiecare clauză C a lui F adăugăm un 3-circuit la G

$$v_C^1, v_C^1 v_C^2, v_C^2, v_C^2 v_C^3, v_C^3, v_C^3 v_C^1;$$

- pentru fiecare variabilă x care apare în formula F , adăugăm un 2-circuit la G

$$v_x, v_x v_{\bar{x}}, v_{\bar{x}}, v_{\bar{x}} v_x, v_x;$$

- pentru fiecare clauză C și fiecare literal u care apare în C adăugăm a G trei arce

$$v_u v_C^1, v_u v_C^2, v_u v_C^3.$$

Exerciții pentru seminarul din săptămâna viitoare

Exercise 8. Considerăm următoarea problemă de decizie
2SAT

Instanță: $\mathcal{C}$ o familie de clauze fiecare cu doi literali.

Întrebare: Există o asignare a valorilor de adevăr către variable astfel ca toate clauzele din $\mathcal{C}$ să fie satisfăcute?

Define G digraful (implicațiilor): $V(G) =$ mulțimea literalilor folosiți în $\mathcal{C}$ și $E(G) = \{\overline{v}_j w_j, \overline{w}_j v_j : C_j = v_j \vee w_j, j = \overline{1, m}\}$ (fiecare clauză introduce în G două arce). Arătați că $\mathcal{C}$ este satisfiabilă dacă și numai dacă x_i și $\overline{x}_i$ aparțin la componente tari conexe diferite ale lui G , $\forall i = \overline{1, n}$. Arătați că această proprietate poate fi verificată în $\mathcal{O}(n + m)$.

Exercise 9. Arătați că următoarea problemă este NP-completă.
MAX-2SAT

Instanță: $\mathcal{C}$ o familie de clauze fiecare cu cel mult doi literali și $k \in \mathbb{N}$.

Întrebare: Există o asignare a valorilor de adevăr către variable astfel ca cel puțin k dintre clauze să fie satisfăcute?

Exercise 10. Considerăm următoarea problemă de decizie NAE-3SAT

Instanță: C o familie de clauze fiecare cu doi literali.

Întrebare: Există o asignare a valorilor de adevăr către variabile astfel ca în fiecare clauză să avem și un literal adevărat și unul fals?

Arătați că următoarea construcție conduce la o reducere polinomială a lui 3SAT la NAE 3SAT (i.e. $3SAT \leq_P \text{NAE } 3SAT$):

- păstrăm variabilele booleene ale instanței 3SAT, $U = \{u_1, u_2, \dots, u_n\}$ și adăugăm o variabilă (nouă) x ;
- pentru fiecare clauză $C_j = v_{j_1} \vee v_{j_2} \vee v_{j_3}$ adăugăm o variabilă nouă y_j și înlocuim C_j cu două clauze:

$$C_j^1 = v_{j_1} \vee v_{j_2} \vee y_j, C_j^2 = v_{j_3} \vee x \vee \overline{y_j}.$$

Exercițiul 11. Considerăm următoarea problemă de decizie MAX-CUT

Instanță: $G = (V, E)$ un graf, $c : E \rightarrow \mathbb{R}$ o funcție de pondere și $k \in \mathbb{R}$.

Întrebare: există o tăietură în G de pondere $\geq k$?

Arătați că următoarea construcție duce la o reducere polinomială a problemei NAE 3SAT la MAXCUT (i.e. $\text{NAE 3SAT} \leq_P \text{MAXCUT}$):

- considerăm o instanță a problemei NAE 3SAT cu clauzele $\mathcal{C} = \{C_1, \dots, C_m\}$ peste mulțimea de variabile booleene $U = \{u_1, u_2, \dots, u_n\}$; putem presupune că fiecare clauză conține trei literal diferiți,
- $V(G) = \{u_i, \bar{u}_i : i = \overline{1, n}\}$ și adăugăm la $E(G)$ muchiile $u_i \bar{u}_i$ de pondere $10m$,
- pentru fiecare clauză $C_j = v_{j_1} \vee v_{j_2} \vee v_{j_3}$ adăugăm la $E(G)$ trei muchii $v_{j_1} v_{j_2}$, $v_{j_2} v_{j_3}$ și $v_{j_1} v_{j_3}$, fiecare de pondere 1,
- $k = 10nm + 2m$.

Memento

- Pentru a arăta că o problemă este NP-completă:
 - I. Mai întâi trebuie arătat mai întâi că este în clasa NP adică un candidat pentru a fi soluție a problemei poate fi verificat în timp polinomial (pentru 3COL: fie o funcție $c : V(G) \rightarrow \mathbb{N}$, se poate verifica că c este o colorare și folosește cel mult trei culori în timp polinomial, de exemplu $\mathcal{O}(n^2)$).
 - II. Apoi urmează demonstrația reducerii polinomiale de la o altă problemă NP-completă (pentru 3COL: 3SAT se reduce în timp polinomial la 3COL).
- Dacă pasul I. lipsește atunci problema de decizie în cauză este NP-hard.

Exercițiul 1. Soluție.

- Mai întâi ar atăm că $\text{INT} \in \text{NP}$: pentru orice $x \in \mathbb{Z}^m$ se poate verifica în $\mathcal{O}(nm)$ dacă $Ax \leq b$ (de ce?).
- Fie $G = (V, E)$ be a graf and $k \in \mathbb{N}$ an instanță for SM.
- Existența unei mulțimi stabile S în G cu $|S| \geq k$ este echivalentă cu existența unei soluții a următoarei probleme liniare

$$(LP) \begin{cases} (1) & \sum_{v \in V} x_v \geq k, \\ (2) & x_u + x_v \leq 1, \quad \forall uv \in E. \\ (3) & x_v \in \{0, 1\}, \quad \forall v \in V(G). \end{cases},$$

unde:

- relațiile (3) definesc x drept vectorul caracteristic al unei submulțimi S a lui $V(G)$,
- relațiile (2) arată că S este o mulțime stabilă,
- iar (1) spune că S conține cel puțin k noduri.

- Fie $V(G) = \{v_1, v_2, \dots, v_n\}$ și $E(G) = \{e_1, e_2, \dots, e_m\}$; putem defini

$$b = (1, 1, \dots, 1, -k, 0, 0, \dots, 0, 1, 1, \dots, 1)^T \in \mathbb{Z}^{2n+m+1}$$

$$A = (a_{ij})_{\substack{1 \leq i \leq 2n+m+1 \\ 1 \leq j \leq n}},$$

$$a_{ij} = \begin{cases} 1, & e_i = v_j v_k, 1 \leq i \leq m \text{ or } n+m+2 \leq i = j \leq 2n+m, \\ -1, & i = m+1 \text{ or } m+2 \leq i = j \leq n+m, \\ 0, & \text{otherwise.} \end{cases}$$

- Construcția lui A și b se face în timpul $\mathcal{O}(n+m)$ (de ce?).
- (LP) are o soluție dacă și numai dacă la întrebarea problemei INT cu A și b drept input răspunsul este afirmativ (de ce?).
- Astfel, $SM \leq_P INT$.

Exercițiul 2. Soluție.

- Fie H un graf și $k \in \mathbb{N}$ - o instanță pentru SM.
- Definim digraful G astfel:

$$V(G) = V(H), E(G) = \{xy, yx : xy \in E(H)\}$$

și $p = |H| - k$ (G se construiește plecând de la H în $\mathcal{O}(|V(H)| + |E(H)|)$); (G și p formează o instanță pentru problema ACYCLIC).

- Dacă H conține o mulțime stabilă S cu $|S| \geq k$, atunci definim $A = V(H) \setminus S$, avem $|A| \leq p$ iar $G \setminus A$ este aciclic (de ce?).
- Dacă G conține o mulțime de noduri A , cu $|A| \leq p$ a. î. $G \setminus A$ este aciclic, atunci $S = V(H) \setminus A$ este o mulțime stabilă în H (de ce?) și $|S| \geq k$.

Exercițiul 3. Soluție.

- Evident, 3-SIMPLE $\in$ NP deoarece, pentru o funcție dată $c : V(H) \rightarrow \{1, 2, 3\}$, se poate verifica în $\mathcal{O}(n^3)$ dacă, pentru orice $e \in E(H)$, $c(u) \neq c(v)$, $\forall u \neq v \in e$. (de ce?)
- Arătăm că 3-COL $\leq_P$ 3-SIMPLE. Fie $G = (V, E(G))$ un graf (input pentru 3-COL).
- Definim o instanță pentru 3-SIMPLE astfel
 $V(H) \leftarrow V(G)$;
 for($[\{x, y, z\}]_G \cong K_3$)
 $E(H) \leftarrow E \cup \{\{x, y, z\}\}$;
 for ($e = xy \in E(G)$ not contained in any K_3) {
 $E(H) \leftarrow E \cup \{\{x, y, w_{xy}\}\}$;
 $V(H) \leftarrow V(H) \cup \{w_{xy}\}$;
 }

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

- Acesta este un algoritm de complexitate polinomială și $\chi(G) \leq 3$ dacă și numai dacă H este simplu (de ce?).

Exercițiul 4. Soluție.

- Ce se întâmplă dacă G nu este conex?
- Dacă $T \in \mathcal{T}_G$, $\Delta(G) \geq \Delta(T)$; ce se întâmplă când $k > \Delta(G)$?
- Presupunem că G este un graf conex și fie $x_0 \in V(G)$ cu $d_G(x_0) = \Delta(G)$ și T rezultatul unei parcurgeri *bfs* a lui G plecând din x_0 , evident $\Delta(T) = ?$; deci răspunsul la întrebarea AGM este ...?.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

- Algoritmul care rezolvă această problemă (și are complexitatea $\mathcal{O}(|V(G)| + |V(G)|)$) este:

```
if( $k > \Delta(G)$ )
```

```
    return ?;
```

```
else {
```

```
    bfs( $G$ ); // în timpul parcurgerii se determină  $\Delta(G)$ ;
```

```
    if( $G$  is not connected)
```

```
        return ?;
```

```
    if( $\Delta(G) > k$ )
```

```
        return ?;
```

```
    else
```

```
        return ?;
```

```
}
```

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 5. Soluție.

(a) Considerăm $V = \{v_1, v_2, \dots, v_n\}$ și următorul algoritm:

$S \leftarrow \emptyset;$

for ($i = 1, n$)

if ($\nexists v_j \in S$ s. t. $v_j v_i \in E$)

$S \leftarrow S \cup \{v_i\};$

for ($i = n, 1$)

if ($i \in S$ și $\exists v_j \in S$ s. t. $v_j v_i \in E$)

$S \leftarrow S \setminus \{v_i\};$

return $S;$

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

- (b) Considerăm o instanță a problemei 3-SAT, $X = \{x_1, x_2, \dots, x_n\}$ - mulțimea variabilelor booleene, $C = \{C_1, C_2, \dots, C_m\}$ o familie de clauze, toate având cel mult trei literali peste X .
- Fie L mulțimea tuturor literalilor care apar în cel puțin o clauză din C . Plecând de la această instanță construim un digraf D :

$$V(D) = \{x_C, y_{C_1}, z_{C_1}, y_{C_2}, z_{C_2}, \dots, y_{C_m}, z_{C_m}\} \cup \left(\bigcup_{j=1}^m \{x_j, \bar{x}_j\} \right).$$

$$E(D) = \left(\bigcup_{i=1}^m \{y_{C_i} x_C, z_{C_i} x_C, z_{C_i} y_{C_i}\} \right) \cup \left(\bigcup_{i=1}^m \{v z_{C_i} : v \in C_i\} \right) \cup \left(\bigcup_{j=1}^n \{x_j \bar{x}_j, \bar{x}_j x_j\} \right).$$

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

- Vom arăta că familia de clauze $\mathcal{C}$ este satisfiabilă dacă și numai dacă D admite un quasi-kernel care conține $x_{\mathcal{C}}$.
- " $\implies$ " Fie $\varphi : X \rightarrow \{0, 1\}$ o asignare a valorilor de adevăr care satisface toate clauzele din $\mathcal{C}$. Definim $S = \{x_{\mathcal{C}}\} \cup \{v \in L : \varphi(v) = 1\}$.
- S este un quasi-kernel în D (de ce?).
- " $\impliedby$ " Fie S be a quasi-kernel containing $x_{\mathcal{C}}$; avem $y_{C_i}, z_{C_i} \notin S$, $\forall i = 1, m$.
- Definim o asignare a valorilor de adevăr $\varphi : X \rightarrow \{0, 1\}$ considerând $\varphi(v) = 1$ dacă și numai dacă $v \in S$.
- Considerăm o clauză C_i , dacă $\varphi(C_i) = 0$, atunci $\varphi(v) = 0$ adică $v \notin S$, pentru orice literal v din C_i . Astfel y_{C_i} și z_{C_i} nu sunt accesibili din S , o contradicție.

Exercițiul 6. Soluție.

- Mai întâi observăm că LPL este în NP (de ce?).
- Apoi arătăm că $TR \leq_P LPL$:
- Fie $G = (V, E)$ un graf (instanță pentru TR). Definim G și $n - 1$ o instanță pentru LPL.
- G este trasabil dacă și numai dacă există un drum în G de lungime cel puțin $n - 1$ (de ce?).

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 7. Soluție.

- Fie X mulțimea de variabile din F ($\overline{X}$ este mulțimea literalilor negativi) și $C_1, C_2, \dots, C_m$ clauzele din F .
- F este satisfiabilă dacă și numai dacă G admite un nucleu.
- “ $\implies$ ” Presupunem că F este o formulă satisfiabilă: există o asignare a valorilor de adevăr $t : X \rightarrow \{0, 1\}$, astfel ca $t(C_j) = 1$, $\forall 1 \leq j \leq m$.
- Fie $S = \{v_u : u \in X \cup \overline{X}, t(u) = 1\}$ - o mulțime stabilă din G .
- Dacă $w \in V(G) \setminus S$, atunci $w = v_{\overline{u}}$ pentru un $u \in S$ sau $w = v_C^i$, pentru o clauză C în care apare cel puțin un literal u ($t(u) = 1$) (de ce?); în amândouă cazurile $v_u w \in E(G)$.
- Astfel, S este un nucleu al lui G .
- “ $\impliedby$ ” Reciproc, presupunem că G are un nucleu S .

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- Pentru orice clauză C din F , $|\{v_C^1, v_C^2, v_C^3\} \setminus S| \geq 2$ și trebuie să existe un nod $w_C = v_C^i \notin S$ care nu este "văzut" din $S \cap \{v_C^1, v_C^2, v_C^3\}$.
- Pentru orice variabilă $x \in X$, $|S \cap \{v_x, \bar{v}_x\}| = 1$ (de ce?), deci putem defini următoarea asignare $t : X \rightarrow \{0, 1\}$

$$t(x) = \begin{cases} 0, & v_{\bar{x}} \in S \\ 1, & v_x \in S \end{cases}.$$

- Fie C o clauză din F ; w_C trebuie să fie văzut și din afara mulțimii $\{v_C^1, v_C^2, v_C^3\}$, deci există un literal u așa ca $v_u \in S$ și $v_u w_C \in E$, astfel u apare în C (de ce?) și $t(u) = t(C) = 1$.
- În plus, G poate fi construit în complexitatea timp (polinomială) $\mathcal{O}(|X| + m)$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 8. Soluție.

- “ $\implies$ ” Presupunem că există $1 \leq i \leq n$ așa încât x_i și $\bar{x}_i$ sunt în aceeași componentă tare conexă a lui G : vor exista în G un $x_i \bar{x}_i$ -drum și un $\bar{x}_i x_i$ -drum

$$x_i = u_0, u_1, \dots, u_k = \bar{x}_i \quad \text{and} \quad \bar{x}_i = v_0, v_1, \dots, v_h = x_i$$

Fie $t : \{x_1, x_2, \dots, x_n\} \rightarrow \{0, 1\}$ o asignare care satisface toate clauzele din $\mathcal{C}$. Fie $t(x_i) = 1$; cum $u_{j-1}u_j \in E(G)$, avem $(\bar{u}_{j-1} \vee u_j) \in \mathcal{C}$, deci $t(u_{j-1}) = 0$ sau $t(u_j) = 1$, pentru orice $1 \leq j \leq k$. Astfel

$$t(u_0) = 1 \implies t(u_1) = 1 \implies \dots \implies t(u_k) = ?$$

- **Ce contradicție se obține de aici?**
- O contradicție similară se obține dacă presupunem că $t(x_i) = 0$ (folosind $\bar{x}_i x_i$ -drumul).

Exerciții rezolvate (parțial)

```
for  $i = 1$  to  $n$  { //  $t$  has not yet been defined anywhere
```

```
     $t[x_i] \leftarrow -1;$ 
```

```
     $t[\overline{x}_i] \leftarrow -1;$ 
```

```
}
```

```
for  $j = p$  down to  $1$  {
```

```
    if  $(\exists u \in V(G_j)$  such that  $t[\overline{u}] \neq -1)$ 
```

```
        for  $v \in V(G_j)$ 
```

```
             $t(v) \leftarrow 1 - t(\overline{u});$ 
```

```
    else
```

```
        for  $v \in V(G_j)$ 
```

```
             $t(v) \leftarrow 1;$ 
```

```
}
```

- Se observă că, dacă $u, v \in V$ sunt într-o aceeași componentă tare conexă a lui G , atunci $\overline{u}$ și $\overline{v}$ sunt de asemenea într-o aceeași componentă tare conexă a lui (de ce? diferită de prima) - componentele tari conexe ale lui G sunt împerecheate.

Exerciții rezolvate (parțial)

- t este bine definită deoarece asignarea este definită global în fiecare component tare conexă și:
 - x_i și $\bar{x}_i$ sunt în component diferite;
 - dacă, pentru $u \in V(G_j)$, $t(\bar{u})$ a fost deja definită, atunci $\bar{u} \in V(G_k)$ cu $k \geq j$ și $\bar{v} \in V(G_k)$, pentru orice $v \in V(G_j)$ (de ce?).

Exercițiul 9. Soluție.

- $\text{MAX 2SAT} \in \text{NP}$ (de ce?). Vom arăta că $3\text{SAT} \leq_P \text{MAX 2SAT}$:
- Fie $U = \{u_1, u_2, \dots, u_n\}$, ($n \in \mathbb{N}^*$), $C = C_1 \wedge C_2 \dots \wedge C_m$ ($m \in \mathbb{N}^*$) with $C_i = v_{i_1} \vee v_{i_2} \vee v_{i_3}$, $i = \overline{1, m}$, (unde $\forall i_j, \exists h \in \{1, 2, \dots, n\}$ s. t. $v_{i_j} = u_h$ or $v_{i_j} = \overline{u_h}$) o instanță of the 3SAT problem.
- Definim următoarea instanță pentru MAX 2SAT: $k = 7m$, $U' = U \cup \{x_1, x_2, \dots, x_m\}$, $C' = \bigwedge_{j=1}^m C'_j$, unde

$$C'_j = x_j \wedge v_{j_1} \wedge v_{j_2} \wedge v_{j_3} \wedge (\overline{v}_{j_1} \vee \overline{v}_{j_2}) \wedge (\overline{v}_{j_2} \vee \overline{v}_{j_3}) \wedge (\overline{v}_{j_3} \vee \overline{v}_{j_1}) \wedge (v_{j_1} \vee \overline{x}_j) \wedge (v_{j_2} \vee \overline{x}_j) \wedge (v_{j_3} \vee \overline{x}_j)$$
- Dacă C este satisfăcută de către o asignare $t : U \rightarrow \{\text{true}, \text{false}\}$, atunci cel puțin un literal din $\{v_{j_1}, v_{j_2}, v_{j_3}\}$ este true, pentru orice $1 \leq j \leq m$.

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

- În toate trei cazurile există o asignare pentru x_j astfel ca exact șapte clauze "mici" (corespunzând lui C_j) să fie true:
 - dacă $t(v_{j_1}) = t(\overline{v}_{j_2}) = t(\overline{v}_{j_3}) = \text{true}$, definim $t(x_j) = \text{false}$;
 - dacă $t(v_{j_1}) = t(v_{j_2}) = t(\overline{v}_{j_3}) = \text{true}$, definim $t(x_j) = \text{false}$;
 - dacă $t(v_{j_1}) = t(v_{j_2}) = t(v_{j_3}) = \text{true}$, definim $t(x_j) = \text{true}$.
- Reciproc, dacă există $7m$ clauze satisfăcute de o asignare, t , pe U' , atunci există cel mult 7 clauze "mici" (corespunzând lui C_j) care sunt satisfăcute:
 - dacă $t(x_j) = \text{false}$, atunci $t(v_{j_k} \vee \overline{x}_j) = \text{true}$, $\forall k = \overline{1, 3}$ și vor mai fi satisfăcute încă trei sau patru (de ce?).
 - dacă $t(x_j) = \text{true}$, atunci $\alpha = |\{k : t(v_{j_k}) = \text{true}, 1 \leq k \leq 3\}|$ este egal cu $\beta = |\{k : t(v_{j_k} \vee \overline{x}_j) = \text{true}, 1 \leq k \leq 3\}|$, și $\alpha + \gamma \in \{3, 4\}$, unde $\gamma = |\{k : t(\overline{v}_{j_k} \vee \overline{v}_{j_h}) = \text{true}, 1 \leq k < h \leq 3\}|$ (de ce?).

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 10. Soluție.

- " $\Rightarrow$ " Presupunem că instanță lui 3SAT este satisfăcută de $t : U \rightarrow \{true, false\}$; atunci $t(x) = true$ și dacă $t(v_{j_1})$ sau $t(v_{j_2})$ sunt $true$, atunci $t(y_j) = false$, altfel $t(y_j) = true$. Răspunsul la NAE-3SAT pentru instanța în cauză va fi afirmative (de ce?).
- " $\Leftarrow$ " Presupunem că instanță lui NAE-3SAT este satisfăcută de $t : U \cup \{x\} \cup Y \rightarrow \{true, false\}$, unde $Y = \{y_1, y_2, \dots, y_m\}$.
- Atunci instanța NAE-3SAT este satisfăcută și de $\bar{t} : U \cup \{x\} \cup Y \rightarrow \{true, false\}$, unde $\bar{t}(v) = \overline{t(v)}$, $\forall v \in U \cup \{x\} \cup Y$.
- Dintre cele două asignări o alegem pe aceea în care x este false. Restricția acestei asignări la U satisface instanța 3SAT (de ce?).

Exercițiul 11. Soluție.

- Construcția lui G se face în $\mathcal{O}(n + m)$.
- Arătăm că $\mathcal{C} = C_1 \wedge C_2 \dots \wedge C_m$ este satisfiabilă așa încât fiecare clauză are și un literal *true* și unul *false* dacă și numai dacă G are o tăietură de pondere cel puțin $10mn + 2m$.
- " $\Rightarrow$ " Presupunem că $\mathcal{C} = C_1 \wedge C_2 \dots \wedge C_m$ is satisfăcută așa încât fiecare clauză are și un literal *true* și unul *false* de către asignarea t .
- Definim următoarea bipartiție pe $V(G)$: $S = \{v : t(v) = \text{false}\}$, $T = V(G) \setminus S = \{v : t(v) = \text{false}\} \cup \{y_1, y_2, \dots, y_m\}$.
- $u_i \bar{u}_i \in A = \{xy \in E(G) : x \in S, y \in T\}$ pentru orice $1 \leq i \leq n$ (de ce?).
- Pentru orice clauză C_j , există două muchii în A : dacă doar v_{j_k} este *true*, atunci $v_{j_k} v_{j_h}, v_{j_k} v_{j_l} \in A$, dacă v_{j_k} și v_{j_h} sunt *true*, atunci $v_{j_k} v_{j_l}, v_{j_h} v_{j_l} \in A$. Toate aceste muchii din A au ponderea $10mn + 2m$.

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

- " $\Leftarrow$ " Fie $A = \{xy \in E(G) : x \in S, y \in T\}$ tăietură de pondere cel puțin $10mn + 2m$.
- Fie $1 \leq i \leq n$, if $u_i \bar{u}_i \notin A$, atunci $c(A) \leq (n-1)10m + 3m < 10nm + 2m$ - contradicție (de ce?).
- Dacă, pentru un j , $|A \cap \{v_{j_1} v_{j_2}, v_{j_2} v_{j_3}, v_{j_3} v_{j_1}\}| = |A_j| \neq 2$, atunci $A_j = 0$ și $c(A) < 10nm + m$ - din nou o contradicție.
- Astfel, pentru orice $1 \leq j \leq m$, există un $k \in \{1, 2, 3\}$ astfel ca $A_j = \{v_{j_k} v_{j_h}, v_{j_k} v_{j_l}\}$.
- Cum (S, T) separă literalii pozitivi de cei negativi, putem defini asignarea: $t(v) = \text{true}$, pentru orice $v \in S$ care satisface toate clauzele (de ce?) și fiecare clauză va conține un literal false și unul true (de ce?).

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *